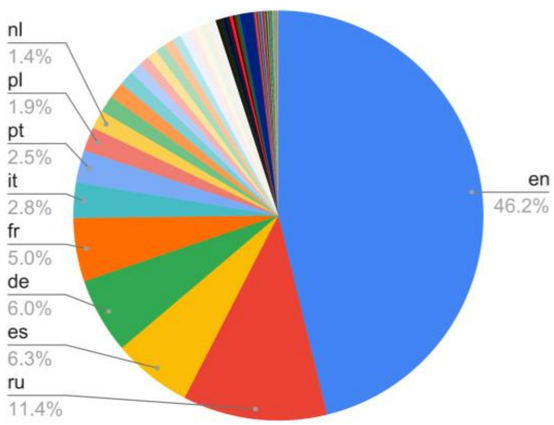
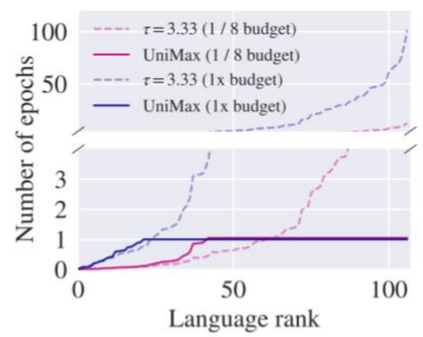
UniMax: Fairer and More Effective Language Sampling for Large-Scale Multilingual Pretraining

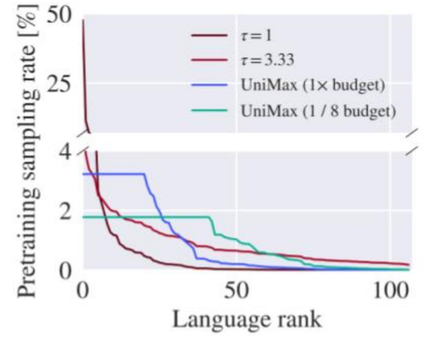
Google Research

Hyung Won Chung, Xavier Garcia, Adam Roberts, Yi Tay, Orhan Firat, Sharan Narang, Noah Constant
h.w.chung27@gmail.com, {nconstant, xgarcia}@google.com

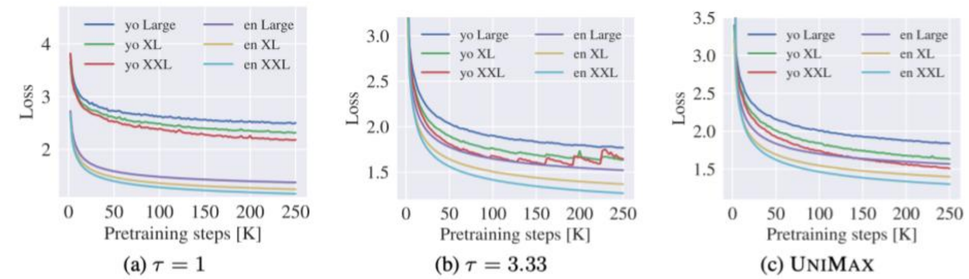




(a) Number of training epochs for each language. Temperature sampling results in a large number of data repeats for low-resource languages, whereas UniMax explicitly caps repeats.



(b) Pretraining sampling distribution. Temperature sampling results in poorly balanced distributions, whereas UniMax provides more uniform distributions without excessive upsampling.

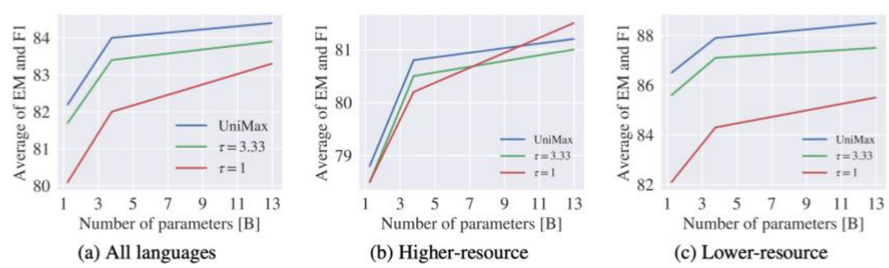


(a) $\tau = 1$ (b) $\tau = 3.33$ (c) UniMax

Figure 2: Pretraining cross-entropy loss on the held-out data over the training steps. With too-low temperature, low-resource languages are sampled too little, and their losses are relatively high. With higher temperature, overfitting becomes more severe with increasing model size.

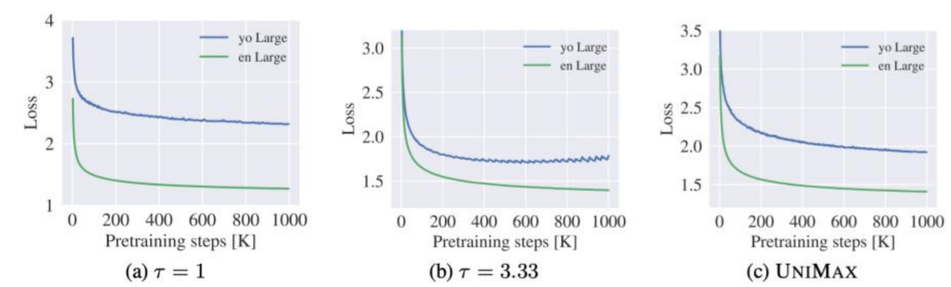
$$p_l = \frac{n_l}{\sum_{l' \in L} n_{l'}} \qquad q_l = \frac{p_l^{1/\tau}}{\sum_{l' \in L} p_{l'}^{1/\tau}}$$

	Temperature (τ)
mBERT (Devlin et al., 2019)	1.43
XLM (Conneau & Lample, 2019)	2.00
XLM-R (Conneau et al., 2020)	3.33
mT5 (Xue et al., 2021)	3.33
XLM-E (Chi et al., 2022)	1.43



(a) All languages (b) Higher-resource (c) Lower-resource

Figure 4: Average TyDi QA GoldP performance across three model sizes. Overall, UniMax outperforms both baselines at all model sizes considered. Breakdowns on higher-resource (top-5) and lower-resource (bottom-4) languages show UniMax outperforms $\tau = 3.33$ on both high- and low-resource, and only underperforms $\tau = 1$ on high-resource at large model scales.



(a) $\tau = 1$ (b) $\tau = 3.33$ (c) UniMax

Figure 3: Pretraining cross-entropy loss on the held-out data over 1M training steps. With the sequence length of 512, this corresponds to $1/2$ character budget. The overfitting behavior emerges only after sufficient number of training steps for $\tau = 3.33$.

Algorithm 1: UniMax

Inputs : Character count c_l of each language l in all the languages L of the training corpus
Total character budget C
The number of epochs per language N
Output: Sampling distribution p_l of each language

```
// Sort the languages by increasing number of character counts
L ← sortByCount(L)
B ← C // Initialize the remaining budget to the total character budget
i ← 0
for l in L do
    b_l ← B / (len(L) − i) // Compute the remaining budget per-language
    if b_l > c_l × N then
        // If per-language budget exceeds N epochs of l, use N epochs
        U_l ← c_l × N
    else
        U_l ← b_l // Otherwise use uniform per-language budget
    end if
    B ← B − U_l // Update the remaining budget
    i ← i + 1
end for
p ← normalize(U)
return p
```

Table 3: Comparison to mT5. XNLI and PAWS-X show average per-language accuracy; the rest show average per-language EM/F1. We use the translate-train setting except for TyDi QA, which uses "in-language". We omit results for the `Large` configuration due to instabilities in training.

	XNLI		PAWS-X		XQuAD		MLQA		TyDi QA	
Model	mT5	umT5	mT5	umT5	mT5	umT5	mT5	umT5	mT5	umT5
Small	72.0	**76.2**	79.9	**87.2**	49.4 / 64.5	**60.5 / 74.0**	38.8 / 56.6	**41.8 / 60.7**	**62.7 / 74.0**	56.6 / 70.0
Base	79.8	**80.8**	89.3	**90.4**	59.7 / 75.3	**67.3 / 79.8**	48.5 / 67.6	**51.6 / 70.5**	68.4 / 79.7	**68.4 / 81.0**
XL	85.3	**86.5**	**91.0**	90.7	56.6 / 75.1	**75.0 / 86.1**	54.5 / 73.5	**58.3 / 76.8**	**78.4 / 87.6**	74.1 / 85.2
XXL	87.1	**87.8**	**91.5**	91.2	71.3 / 85.2	**77.9 / 88.2**	57.4 / 76.0	**70.5 / 78.6**	79.5 / 88.7	**81.2 / 89.7**

Resources

- mC4 3.1.0, available via HuggingFace
- umT5 checkpoints, available via T5X repo